

Informe de entrega - Proyecto 3 PLE

Autor: Jorge Sebastián Otálora Bernal

Código: 202312287

Objetivo

El objetivo de esta entrega fue continuar la definición de VeriLang implementada en Rascal y convertirla en un lenguaje ejecutable con parser, AST, generación de salida, anotaciones de tipos y validaciones semánticas.

Alcance implementado

Se implementaron los siguientes elementos:

- gramática concreta de VeriLang en `Syntax.rsc`;
- AST propio en `AST.rsc`;
- parser de archivos `.vl` en `Parser.rsc`;
- conversión del parse tree al AST en `ToAST.rsc`;
- generador de salida en consola desde el AST en `Generator.rsc`;
- validaciones semánticas y de tipos en `TypeChecker.rsc`;
- integración inicial con TypePal en `Checker.rsc`;
- plugin para registrar archivos `.vl` en `Plugin.rsc`;
- ejemplos de entrada en la carpeta `instance/`.

Cambios frente a iteraciones anteriores

La gramática fue ajustada para que cargue correctamente en Rascal 0.42.1 y para corregir problemas detectados en la base anterior:

- se uso un nombre de proyecto valido para Rascal: `js-otalorab12-project-verilang`;
- se cambió el símbolo inicial a `Program`;
- se evito usar `Module` como nombre de dato del AST para no chocar con el concepto de modulo de Rascal;
- se separó el parser, la conversión a AST, el generador y el chequeador en módulos independientes;
- se estandarizó la extensión de entrada como `.vl`;
- se agregó soporte para `defdata`, usado para estructuras de datos con tipo declarado por el usuario;
- se agregó soporte para valores anotados como `1:Int`, `true:Bool`, `'a':Char`, `"txt":String` y `1.5:Real`.

Validaciones

El chequeador implementado revisa:

- que los tipos usados en operadores, variables y estructuras existan como espacios o tipos base;
- que el padre de un espacio ordenado exista;
- que las variables usadas en expresiones y ecuaciones esten declaradas;
- que los operadores usados en reglas esten declarados;
- que los elementos referenciados dentro de una estructura `defdata` existan;
- que los elementos referenciados dentro de una estructura `defdata` correspondan con el tipo declarado de esa estructura;
- que los literales anotados correspondan con su tipo real y con el tipo declarado de la estructura.

Generación de salida

El archivo `Main.rsc` carga `instance/spec1.vl`, construye el AST, genera una representación textual del programa y muestra el resultado en consola.

El archivo `CheckExamples.rsc` ejecuta los ejemplos incluidos:

- `spec1.vl`, ejemplo válido general;
- `spec_typed_values.vl`, ejemplo válido de valores anotados;
- `spec_error.vl`, ejemplo con errores intencionales.

Uso de fuentes externas

Se usaron como guía los siguientes materiales:

- `ple_project_2.pdf`: se usó para confirmar que la iteración anterior pedía gramática concreta, resaltado de sintaxis y AST.
- `ple_project_3.pdf`: se usó como enunciado principal de esta entrega, especialmente para parser, generación, TypePal, anotaciones de tipos y validación de estructuras.
- `rascal_tutorial.pdf`: se usó como referencia para el flujo de trabajo en Rascal: parser, generador, plugin y TypePal.
- `Solucion_propuesta_PLE_proyecto_1_202610.pdf`: se usó como guía conceptual de la gramática base de VeriLang: módulos, `using`, espacios, operadores `curry`, variables, reglas, expresiones, ecuaciones y atributos.
- proyecto de referencia compartido por las compañeras Isabella Salcedo Delgadillo y Valentina Rojas Forero: se usó como guía de organización de módulos Rascal (Syntax, AST, Parser, ToAST, Generator, TypeChecker) y como punto de comparación para pruebas.
- repositorios de ejemplo compartidos por el profesor: se revisaron para confirmar la organización de ejemplos y ejecución, sin copiar funcionalidades ajenas al alcance de VeriLang.

La solución entregada no es una copia directa de esas fuentes. Se realizaron ajustes propios en la gramática, el AST, el conversor a AST, el generador, las validaciones, los ejemplos y la documentación. En particular, se corrigieron nombres, rutas de proyecto, extensión `.vl`, manejo de estructuras `defdata`, valores anotados y reglas de validación.

Cómo ejecutar

La forma recomendada es ejecutar el proyecto desde VS Code con la extensión de Rascal:

1. abrir `src/main/rascal/Main.rsc`;
2. ejecutar la acción `Run in new Rascal terminal` sobre la función `runDefault`;
3. revisar en consola la salida generada para `instance/spec1.vl` y el resultado del chequeo de tipos.

Para correr todos los ejemplos, abrir `src/main/rascal/CheckExamples.rsc` y ejecutar `runExamples`.

También puede hacerse desde una terminal de Rascal. Para ejecutar el ejemplo principal:

```
import Main;
runDefault();
```

Para ejecutar cualquier archivo `.vl` del proyecto:

```
import Main;
runFile(|project://js-otalorab12-project-verilang/instance/spec1.vl|);
```

Para validar todos los ejemplos incluidos:

```
import CheckExamples;
runExamples();
```

Como alternativa, desde la raíz del proyecto se puede ejecutar con la consola del sistema:

```
java -jar "$env:USERPROFILE\.m2\repository\org\rascalmpl\rascal\0.42.1\rascal-0.42.1.jar" Main
```

Para correr los ejemplos de validación desde la consola del sistema:

```
java -jar "$env:USERPROFILE\.m2\repository\org\rascalmpl\rascal\0.42.1\rascal-0.42.1.jar"
CheckExamples
```

Archivos no incluidos en la entrega

La carpeta `referencias/` no debe incluirse en el ZIP final, porque contiene PDFs, capturas y proyectos usados como material de consulta. La carpeta `target/` tampoco debe incluirse, porque contiene artefactos generados por Maven.